RASCUNHO INTEGRAL PRODUZIDO COM IA

Identificação pessoal não preenchida

ALERTA DE ALAGAMENTOS

Documentação técnica em formato acadêmico — versão para revisão

LOCAL NÃO INFORMADO

2026

IDENTIFICAÇÃO NÃO PREENCHIDA

ALERTA DE ALAGAMENTOS

Documentação do protótipo Alerta de Alagamentos, organizada para Atividade Prática Supervisionada. Versão integral produzida com auxílio de inteligência artificial; sua utilização depende da regra de autoria e da orientação do professor. Identificação pessoal omitida conforme solicitado.

LOCAL NÃO INFORMADO

2026

FOLHA DE APROVAÇÃO – MODELO

ALERTA DE ALAGAMENTOS

[Preencher natureza e dados reais se esta folha for exigida para a APS.]

Aprovado em: ____ / ____ / _____

[Banca / avaliadores: preencher apenas conforme orientação institucional.]

_____

[Avaliador e instituição]

_____

[Avaliador e instituição]

RESUMO

O projeto Alerta de Alagamentos é um protótipo acadêmico para registrar e consultar relatos de alagamentos urbanos por meio de um aplicativo móvel conectado a uma API. O objetivo é permitir que contas diferentes compartilhem registros mantidos em um servidor, com localização e mídia opcionais. A implementação utiliza arquitetura cliente-servidor, comunicação HTTP, JSON e multipart, autenticação por sessão e persistência em SQLite e arquivos privados. O aplicativo permite criar conta, entrar, consultar lista ou mapa, cadastrar, abrir detalhes e resolver relatos pela conta autora. A análise técnica considera autenticação, autorização, transações, concorrência e falhas parciais. Os registros disponíveis documentam 56 testes da API aprovados e 15 cenários integrados web aprovados, com dados fictícios, GPS simulado e arquivos PNG e MP4 de teste. Esses resultados abrangem as operações exercitadas e a persistência após reiniciar a API. A execução em dois celulares com Expo Go, os recursos nativos e o mapa base online permanecem pendentes de validação física. O protótipo conserva armazenamento centralizado, atualização manual e dependência da rede. Os relatos são informações dos participantes, sem verificação oficial. A versão integral deste documento foi produzida com auxílio de inteligência artificial e deve ser avaliada pelo grupo e pelo professor segundo a regra de autoria recebida.

Palavras-chave: sistemas distribuídos; aplicativo móvel; relatos de alagamentos; persistência.

ABSTRACT

Alerta de Alagamentos is an academic prototype for registering and consulting user reports of urban flooding through a mobile application connected to an API. Its objective is to allow different accounts to share records stored on a server, with optional location and media. The implementation uses a client-server architecture, HTTP communication, JSON and multipart representations, session authentication, and persistence in SQLite and private files. Users can register an account, sign in, consult a list or map, create reports, view details, and resolve reports through the author's account. The technical analysis addresses authentication, authorization, transactions, concurrency, and partial failures. The available records document 56 successful API tests and 15 successful integrated web scenarios using fictitious data, simulated geolocation, and PNG and MP4 test files. These results cover the exercised operations and persistence after restarting the API. Execution on two physical phones using Expo Go, native capabilities, and online base maps still requires physical validation. Storage remains centralized, updates are requested manually, and operation depends on connectivity. Reports are participant statements without official verification. This draft was produced with artificial intelligence assistance and requires review under the stated authorship requirements.

Keywords: distributed systems; mobile application; flood reports; persistence.

LISTA DE FIGURAS

Figura 1 – Diagrama de casos de uso.....21
Figura 2 – Diagrama de classes conceituais.....22
Figura 3 – Diagrama de sequência.....23
Figura 4 – Diagrama de atividades.....24
Figura 5 – Diagrama entidade-relacionamento físico.....25

SUMÁRIO

1 OBJETIVO E MOTIVAÇÃO.....8
2 INTRODUÇÃO.....9
2.1 Contexto e delimitação.....9
2.2 Visão do trabalho e organização.....10
3 FUNDAMENTOS.....11
3.1 Sistemas distribuídos e middleware.....11
3.2 Comunicação e representação.....12
3.3 Mobilidade, localização e mapas.....13
3.4 Segurança, persistência e falhas.....14
4 PLANO DE DESENVOLVIMENTO.....15
4.1 Escopo e requisitos.....15
4.2 Elementos, ferramentas e ambiente.....16
4.3 Etapas e integração.....17
4.4 Verificação e evidências.....18
4.5 Riscos, backup e demonstração.....19
5 PROJETO DO PROGRAMA.....20
5.1 Requisitos e módulos.....20
5.2 Diagrama de casos de uso.....21
5.3 Diagrama de classes conceituais.....22
5.4 Diagrama de sequência.....23
5.5 Diagrama de atividades.....24
5.6 Diagrama entidade-relacionamento físico.....25
6 RELATÓRIO DE CÓDIGO.....26
6.1 Trechos selecionados – parte 1.....27
6.2 Trechos selecionados – parte 2.....28
6.3 Trechos selecionados – parte 3.....29
6.4 Trechos selecionados – parte 4.....30
6.5 Trechos selecionados – parte 5.....31
6.6 Trechos selecionados – parte 6.....32
7 CONSIDERAÇÕES FINAIS.....33
7.1 Apresentação do programa.....33
REFERÊNCIAS.....34

APÊNDICE A – ORIENTAÇÕES PARA FICHAS DE APS.....35

1 OBJETIVO E MOTIVAÇÃO

O objetivo geral do projeto Alerta de Alagamentos é disponibilizar um aplicativo móvel que permita cadastrar e consultar relatos de alagamentos urbanos por meio de um serviço compartilhado. A proposta reúne autenticação, descrição textual, localização opcional, visualização em mapa e um anexo de imagem ou vídeo. O resultado pretendido é que uma conta registre uma situação e outra conta, em dispositivo distinto, consulte o mesmo registro mantido pelo servidor.

A motivação do protótipo está na organização de informações que diferentes participantes desejem compartilhar sobre condições observadas em uma via. O relato reúne bairro, endereço, descrição e gravidade informada pelo autor, evitando que esses elementos permaneçam dispersos em registros locais de cada aparelho. A solução oferece uma representação comum, acessível mediante autenticação, com identificação do autor e data definida pelo servidor. Os relatos expressam informações dos participantes e não correspondem a alertas oficiais ou ocorrências verificadas.

Os objetivos específicos são validar os dados recebidos, armazenar registros e arquivos de forma persistente, apresentar os detalhes e restringir a alteração de situação à conta autora. Também se pretende exercitar conceitos de comunicação cliente-servidor, representação de dados, autenticação, autorização, transações e falhas parciais. GPS e anexos são opcionais para que a ausência de permissão ou de arquivo não impeça o cadastro textual.

O critério móvel de conclusão é o cadastro em um celular, a conservação do dado no servidor e a consulta em outro dispositivo. A implementação possui registros de testes automatizados locais e entre clientes web. A execução desse critério em dois celulares com Expo Go, incluindo recursos nativos, continua pendente de validação física.

2 INTRODUÇÃO

2.1 Contexto e delimitação

O acesso a aplicações por dispositivos móveis permite que informações sejam registradas no local em que uma situação é observada e posteriormente consultadas em outros aparelhos. Para que esse compartilhamento tenha um comportamento compreensível, a aplicação precisa definir quais dados recebe, onde os conserva, quem pode alterá-los e como informa uma falha de comunicação. Essas decisões formam o contexto técnico do projeto Alerta de Alagamentos.

O tema escolhido é o registro de relatos de alagamentos urbanos. O sistema organiza informações sobre uma situação descrita pelo participante, sem realizar confirmação técnica da ocorrência. A gravidade é declarada pelo autor e a situação pode ser alterada por ele para resolvida. Não há integração com órgãos públicos, sensores ambientais, emissão de alertas oficiais ou processo de verificação da veracidade do conteúdo. Essa delimitação orienta tanto a interface quanto a interpretação dos resultados apresentados.

A proposta utiliza um aplicativo como cliente de uma API executada em um computador. A API centraliza as regras de autenticação, validação e persistência. Os registros permanecem em SQLite, enquanto as mídias são armazenadas em uma pasta privada e relacionadas ao banco por metadados. Outros clientes acessam o conteúdo por requisições à mesma API, o que permite estudar a comunicação entre processos localizados em ambientes distintos.

O trabalho considera um contexto acadêmico de desenvolvimento e demonstração. A apresentação pelo Expo Go mantém o servidor da API e o servidor de desenvolvimento do aplicativo em execução. A rede deve permitir que os aparelhos alcancem ambos os serviços. Os registros de teste usam dados fictícios; não foram realizados levantamento de campo, entrevistas ou avaliação com uma amostra de usuários. O contexto descrito fundamenta a construção do protótipo, sem atribuir a ele eficácia operacional em emergências.

2.2 Visão do trabalho e organização

O protótipo oferece uma tela de acesso e criação de conta, uma listagem com alternativa de mapa, um formulário e uma tela de detalhes. No cadastro, o participante informa bairro, endereço, descrição e gravidade. A localização pode ser obtida por uma ação explícita, com permissão em primeiro plano. Também é possível selecionar um arquivo JPEG, PNG ou MP4, respeitando os limites definidos. A associação entre relato e mídia é confirmada pelo servidor.

A consulta depende de sessão válida. Qualquer conta autenticada pode visualizar os relatos compartilhados e seus anexos, mas somente o autor pode alterar a situação do próprio registro. Essa regra é aplicada pela API, além de orientar a apresentação dos botões. O identificador e a data são gerados pelo servidor, e a autoria é obtida da sessão, evitando que o cliente escolha livremente esses valores.

O desenvolvimento foi organizado de modo incremental: funcionamento básico da API, modelagem e persistência, operações de relatos, interface móvel e ampliação com autenticação, localização e mídia. Os capítulos seguintes distinguem os conceitos de sistemas distribuídos das ferramentas usadas na implementação. Depois são apresentados o plano de desenvolvimento, os requisitos e os modelos do programa, seguidos de trechos de código e considerações sobre os resultados.

A atualização entre clientes ocorre quando uma nova consulta é realizada. O aplicativo apresenta estados de carregamento, conteúdo, lista vazia e erro. Durante falhas de envio, os dados digitados são preservados para permitir nova tentativa. O conjunto foi projetado para demonstrar um fluxo compartilhado com armazenamento centralizado; disponibilidade contínua, uso sem rede, replicação e moderação não integram a implementação atual. Esses limites são considerados na interpretação das evidências e no planejamento da demonstração.

3 FUNDAMENTOS

3.1 Sistemas distribuídos e middleware

Um sistema distribuído envolve componentes que executam em processos ou dispositivos distintos e precisam cooperar por comunicação. No projeto, o aplicativo e a API constituem processos separados: o cliente solicita uma operação e o servidor responde segundo um contrato. A distribuição da execução traz dependência de rede, atrasos e possibilidade de que apenas parte do conjunto fique indisponível. Assim, uma tela aberta não garante que o serviço remoto esteja acessível.

A arquitetura adotada é cliente-servidor. O cliente recebe interações e representa o estado da interface; o servidor autentica, valida e conserva os dados compartilhados. Essa separação permite que diferentes aparelhos utilizem a mesma regra de negócio. A aplicação não fornece acesso direto do celular ao arquivo SQLite: o conteúdo é consultado e alterado por rotas da API, com respostas em formatos definidos.

Middleware pode ser entendido, no contexto desta análise, como a camada de intermediação que oferece mecanismos de comunicação e integração entre componentes. A mediação inclui representar uma solicitação, transportá-la, encaminhá-la à operação correspondente e devolver um resultado interpretável. No protótipo, o contrato HTTP, a serialização e o processamento das requisições compõem essa intermediação. O framework empregado implementa parte desses mecanismos; a fundamentação trata do papel arquitetural, enquanto a escolha de ferramentas é discutida no plano.

A distribuição do cliente e do serviço convive com armazenamento centralizado. Há um computador responsável pela API, pelo banco e pelos uploads. A coordenação entre vários clientes não constitui replicação do banco, consenso ou tolerância a falhas. Quando esse servidor ou sua rede falha, as operações dependentes deixam de concluir. A aplicação trata essa condição apresentando erro e permitindo nova consulta após a recuperação, mantendo explícitas as garantias limitadas do protótipo.

3.2 Comunicação e representação

HTTP define a semântica das interações utilizadas entre aplicativo e API. Uma requisição indica método, recurso, cabeçalhos e, quando necessário, um corpo. A resposta informa um código de estado e uma representação. No projeto, GET consulta recursos, POST cria contas, sessões e relatos, e PATCH atualiza a situação de uma ocorrência. Essa organização aproxima cada operação de uma finalidade reconhecível no contrato (Fielding; Nottingham; Reschke, 2022).

Os códigos HTTP ajudam a separar resultados. Uma criação confirmada retorna 201; consultas bem-sucedidas retornam 200. Dados inválidos podem produzir 400, ausência de sessão válida resulta em 401 e uma alteração feita por outra conta é recusada com 403. O cliente precisa verificar o estado da resposta: receber uma resposta HTTP de erro não equivale a uma falha de transporte. Por isso, a comunicação da interface verifica o resultado antes de considerar uma operação concluída.

JSON representa objetos, listas, textos, números, valores lógicos e ausência de valor em uma sintaxe independente da linguagem (Bray, 2017). Essa característica permite representar o relato em JavaScript e processá-lo em Python. Quando há anexo, o envio utiliza multipart/form-data, que separa campos e arquivo em partes de uma mesma mensagem, conforme o formato descrito por Masinter (2015). O servidor valida tanto os campos quanto o conteúdo da mídia.

A comunicação tem prazo de espera definido. Um temporizador evita carregamento indefinido quando a API demora a responder. Contudo, um prazo encerrado não comprova que o servidor deixou de gravar: a confirmação pode se perder depois do commit. O bloqueio do botão reduz toques repetidos durante um envio, mas não oferece idempotência para todas as falhas. A orientação operacional é consultar a lista antes de repetir um cadastro cuja confirmação não chegou.

3.3 Mobilidade, localização e mapas

Dispositivos móveis combinam interação por toque, diferentes tamanhos de tela, recursos locais e conectividade variável. Essas características influenciam o tamanho dos controles, o uso do teclado, a rolagem e a apresentação de estados de espera. A mobilidade também exige distinguir o armazenamento local do estado mantido pelo serviço: informações que aparecem na tela podem estar desatualizadas até que outra consulta seja concluída.

A localização do aparelho pode ser obtida pelos mecanismos de posicionamento disponibilizados pelo sistema operacional. A precisão e a disponibilidade dependem do dispositivo, do ambiente e das permissões concedidas. O protótipo solicita a posição somente quando a pessoa aciona o botão correspondente. Não há acompanhamento contínuo nem coleta em segundo plano. A ausência de permissão permite continuar o cadastro textual, conservando a natureza opcional desse recurso.

Latitude e longitude representam a posição compartilhada. A API exige que as duas coordenadas sejam informadas juntas e valida seus intervalos e sua finitude. O mapa apresenta apenas relatos que possuem coordenadas válidas. Essa representação permite relacionar os registros ao espaço, mas não verifica se a pessoa estava no endereço descrito. A interface orienta que o GPS seja utilizado quando o participante estiver no local do relato.

Mapas dependem de dados e serviços de representação cartográfica. No aplicativo nativo são utilizados os provedores disponíveis pela biblioteca compatível com Expo Go. Na web, o mapa usa Leaflet e mosaicos OpenStreetMap, com atribuição. Os marcadores representam os dados do serviço; o mapa base depende de conectividade externa. Quando o carregamento dos mosaicos falha, a listagem e as informações textuais permanecem como caminhos de consulta. Na validação automatizada, as coordenadas foram simuladas e os mosaicos foram bloqueados deliberadamente para exercitar esse comportamento.

3.4 Segurança, persistência e falhas

Autenticação identifica uma conta; autorização determina o que essa conta pode fazer sobre um recurso. O projeto utiliza senha para abrir a sessão e um token aleatório para autenticar os pedidos seguintes. Antes de alterar a situação, a API compara a autoria do relato com a conta da sessão. Essa verificação no servidor impede que a simples manipulação da interface conceda permissão de alteração.

As senhas são armazenadas por hash scrypt com salt, e o servidor conserva apenas o hash SHA-256 do token de sessão. A validade é limitada e o logout revoga a sessão. No cliente nativo, o token é mantido no SecureStore; na versão web, permanece somente em memória. O transporte HTTP local é destinado à demonstração controlada. A disponibilização pública exigiria HTTPS e preparação do serviço, pois a proteção do armazenamento não elimina a necessidade de proteger o tráfego.

Persistência é a conservação dos dados além do tempo de execução de uma tela ou processo. SQLite mantém contas, sessões, relatos e metadados, enquanto os arquivos de mídia ficam em armazenamento separado. Transações agrupam alterações que devem ser confirmadas ou desfeitas. Na operação com anexo, uma falha na gravação do banco leva também à remoção compensatória do arquivo preparado, reduzindo o risco de conteúdo órfão.

Concorrência permite que clientes enviem pedidos em períodos sobrepostos. O banco coordena o acesso ao arquivo e serializa as gravações. Não foi realizado teste de carga para estabelecer capacidade. Já a falha parcial ocorre quando a interface continua disponível e a API deixa de responder. A aplicação informa o erro, preserva o formulário e permite tentar novamente. Essas medidas melhoram a experiência diante da falha, mantendo a dependência de um servidor central e da rede.

4 PLANO DE DESENVOLVIMENTO

4.1 Escopo e requisitos

O plano parte de um fluxo mínimo: consultar relatos, cadastrar um registro, abrir seus detalhes e alterar sua situação. A ampliação de escopo incorporou contas, sessões, autoria, localização opcional, mapa e um anexo por ocorrência. Essas decisões se refletem em requisitos observáveis, associados a rotas da API, componentes da interface e regras de persistência.

O requisito de cadastro define quatro textos obrigatórios e valores controlados para gravidade. Identificador, data, situação inicial e autor são definidos no servidor. A consulta deve apresentar registros compartilhados para contas autenticadas, ordenados do mais recente para o mais antigo. A atualização de situação é restrita ao autor. A localização e a mídia permanecem opcionais, para que sua indisponibilidade não impeça a descrição textual do relato.

As restrições incluem limites de comprimento, formatos de mídia, tamanho do arquivo e validação das coordenadas. Um cadastro com anexo deve concluir a associação entre dados e arquivo ou responder com falha sem deixar um registro parcial. O cliente apresenta o estado de envio e mantém os campos quando recebe erro. Essas regras são consideradas em conjunto, pois a experiência da tela depende das garantias oferecidas pela API.

A aceitação funcional requer comunicação entre dispositivos e persistência após reiniciar o serviço. O planejamento da validação separa verificações automatizadas locais de testes em aparelhos físicos. Os requisitos não incluem chat, notificações, rastreamento contínuo, operação offline ou moderação. Essa delimitação mantém o desenvolvimento concentrado nos recursos efetivamente implementados e permite demonstrar o caminho completo dos dados.

4.2 Elementos, ferramentas e ambiente

A implementação utiliza Python no servidor e JavaScript na interface. Flask organiza as rotas e o tratamento das requisições; React Native fornece componentes móveis; Expo apoia a execução no ambiente de desenvolvimento. Essas escolhas são ferramentas para implementar a arquitetura, os protocolos e os mecanismos estudados na fundamentação. SQLite foi adotado para persistência em arquivo, evitando a instalação de um serviço de banco separado para a demonstração.

O ambiente documentado utiliza Python compatível com as dependências, a partir da versão 3.11, e teve Node.js 24.22.1 nos testes. O aplicativo está baseado no Expo SDK 57, React 19.2.3 e React Native 0.86.3. As versões ficam registradas nos manifests e lockfiles. A instalação reproduzível usa o arquivo de requisitos do backend e npm ci no aplicativo, preservando a combinação de pacotes definida pelo projeto.

Durante o desenvolvimento, API e Expo executam como serviços distintos. O Flask recebe os pedidos na porta 5000; o Metro fornece o aplicativo na porta 8081. Para usar celulares na rede local, ambos precisam estar alcançáveis e o endereço da API deve apontar para o IPv4 do computador. O endereço localhost representa o próprio dispositivo e não deve ser confundido com o servidor acessível pelo celular.

A edição do código pode ser realizada em um editor como VS Code, e o Git registra as versões. O ambiente virtual Python mantém as bibliotecas do servidor separadas. Permissões de localização e seleção de mídia são solicitadas no aplicativo conforme a ação do participante. O planejamento considera também os serviços externos dos mapas e a necessidade de conferir a compatibilidade do Expo Go instalado nos aparelhos da apresentação.

4.3 Etapas e integração

A organização do desenvolvimento segue dependências entre componentes. O primeiro ponto é executar a rota de saúde, verificando que o servidor responde antes de envolver o banco. Em seguida, a estrutura de dados é criada por um inicializador idempotente. As operações de lista, cadastro, detalhes e situação permitem validar o contrato da API independentemente da interface móvel.

A etapa da interface utiliza funções de comunicação separadas das telas. A listagem apresenta carregamento, vazio, erro e conteúdo. O formulário valida os campos, controla o envio e conserva as informações em falhas. A tela de detalhes consulta o identificador selecionado e reflete alterações confirmadas no servidor. Essa separação reduz a duplicação das regras de comunicação e facilita observar erros de integração.

A ampliação com autenticação exige alterar o contrato antes de liberar as ações na interface. O token passa a identificar a conta e a autorização passa a depender da autoria. A migração mantém registros antigos sem atribuí-los automaticamente a usuários novos. Depois são integrados os campos opcionais de localização e mídia, com validação e persistência correspondentes no servidor.

O histórico disponível registra incrementos no código e na documentação; ele não fornece, por si só, as horas ou a participação individual de alunos. O plano de 21 dias apresentado como referência não deve ser convertido em diário realizado. Responsáveis, datas de trabalho e esforço precisam ser registrados por cada integrante com base na atividade real. A integração técnica pode ser descrita a partir do código, enquanto os registros acadêmicos individuais permanecem sujeitos à confirmação dos participantes.

4.4 Verificação e evidências

A estratégia de verificação combina testes da API e um fluxo integrado da interface web. Os testes da API utilizam bancos temporários e incluem autenticação, autorização, limites de campos, coordenadas, arquivos, migração e comportamento em falhas. Essa abordagem exercita a regra do servidor sem depender de interações manuais para cada caso.

O registro técnico da versão ampliada documenta 56 testes da API aprovados e 15 cenários integrados aprovados. O fluxo integrado usa duas contas e contextos isolados de Chromium, arquivos PNG e MP4 reais de teste e uma API com armazenamento temporário. Foram verificados login, cadastro, leitura protegida de mídia, reprodução de vídeo, bloqueio de alteração por outra conta, logout, preservação de formulário e recuperação após reiniciar a API.

A geolocalização desse ensaio foi simulada pelo navegador. Os mosaicos OpenStreetMap foram bloqueados de propósito para observar o comportamento alternativo da interface. Portanto, os resultados apoiam a validação das coordenadas, dos marcadores e do tratamento de erro, sem comprovar o sensor GPS ou o mapa nativo de um celular. Também foram registradas checagem de tipos, lint e exportação de bundles para web, Android e iOS.

A próxima validação física deve utilizar contas distintas em dois aparelhos. Um usuário cadastra, o outro consulta e tenta alterar, a conta autora resolve e a outra atualiza a lista. Devem ser registrados os resultados com GPS permitido e negado, imagem, vídeo e reinicialização do servidor. Aparelhos, versões, data e evidências precisam acompanhar essa execução. A rastreabilidade entre requisito, teste e resultado evita atribuir aos testes locais uma abrangência que eles não possuem.

4.5 Riscos, backup e demonstração

O planejamento de operação considera a dependência de um servidor central, da rede e dos provedores de mapa. Uma falha pode impedir a abertura do aplicativo pelo Expo, a consulta à API ou apenas o carregamento do mapa base. O diagnóstico deve identificar qual serviço deixou de responder antes de alterar configurações. A rota de saúde ajuda a separar o acesso à API dos demais componentes.

No teste em rede local, é necessário conferir o IPv4 do computador, o endereço configurado no aplicativo e as permissões do Python e do Node na rede privada. A permissão de um processo não libera automaticamente o outro. O procedimento mantém o firewall ativo e verifica as portas usadas. Na web, a geolocalização depende de um contexto seguro, como HTTPS ou localhost; essa condição deve ser considerada ao interpretar um erro de permissão.

O backup deve conservar o banco e a pasta de uploads de maneira consistente. Para a demonstração, o procedimento documentado é parar a API e copiar ambos para um local separado. Copiar somente o SQLite preserva os metadados, mas não os arquivos associados. O código é mantido no Git, enquanto banco, anexos de participantes e configurações locais ficam fora do versionamento.

A apresentação deve ser ensaiada com o mesmo fluxo previsto para a avaliação. Todos os integrantes precisam participar e explicar o caminho entre aplicativo, comunicação, servidor e armazenamento. Os vídeos de apoio são identificados como testes web com dados fictícios. A conclusão da demonstração em aparelhos dependerá da execução e do registro físico, e os formulários acadêmicos deverão refletir as atividades e horas reais de cada aluno.

5 PROJETO DO PROGRAMA

5.1 Requisitos e módulos

O programa está organizado em interface, comunicação, serviço e armazenamento. A interface controla acesso, lista, mapa, formulário e detalhes. O serviço de comunicação concentra requisições, sessão, tempo limite e preparação de arquivos. No servidor, as rotas aplicam autenticação, validação e autorização antes de consultar ou alterar o banco.

Os requisitos de conta abrangem cadastro, login, consulta de sessão e logout. Os requisitos de relatos abrangem criar, listar, detalhar e resolver com verificação de autoria. Os complementos opcionais incluem obter uma posição e anexar um arquivo, mantendo o envio textual disponível quando esses recursos não forem usados. A consulta de anexo também exige sessão válida.

A matriz relaciona operações a módulos existentes e a critérios verificáveis. O modelo de classes é conceitual, pois as entidades do domínio são representadas por tabelas, registros e funções, sem classes Python correspondentes a cada entidade. O DER representa as restrições físicas do schema, enquanto a documentação também informa regras mais restritivas aplicadas pela API.

Quadro 1 – Rastreabilidade dos requisitos implementados

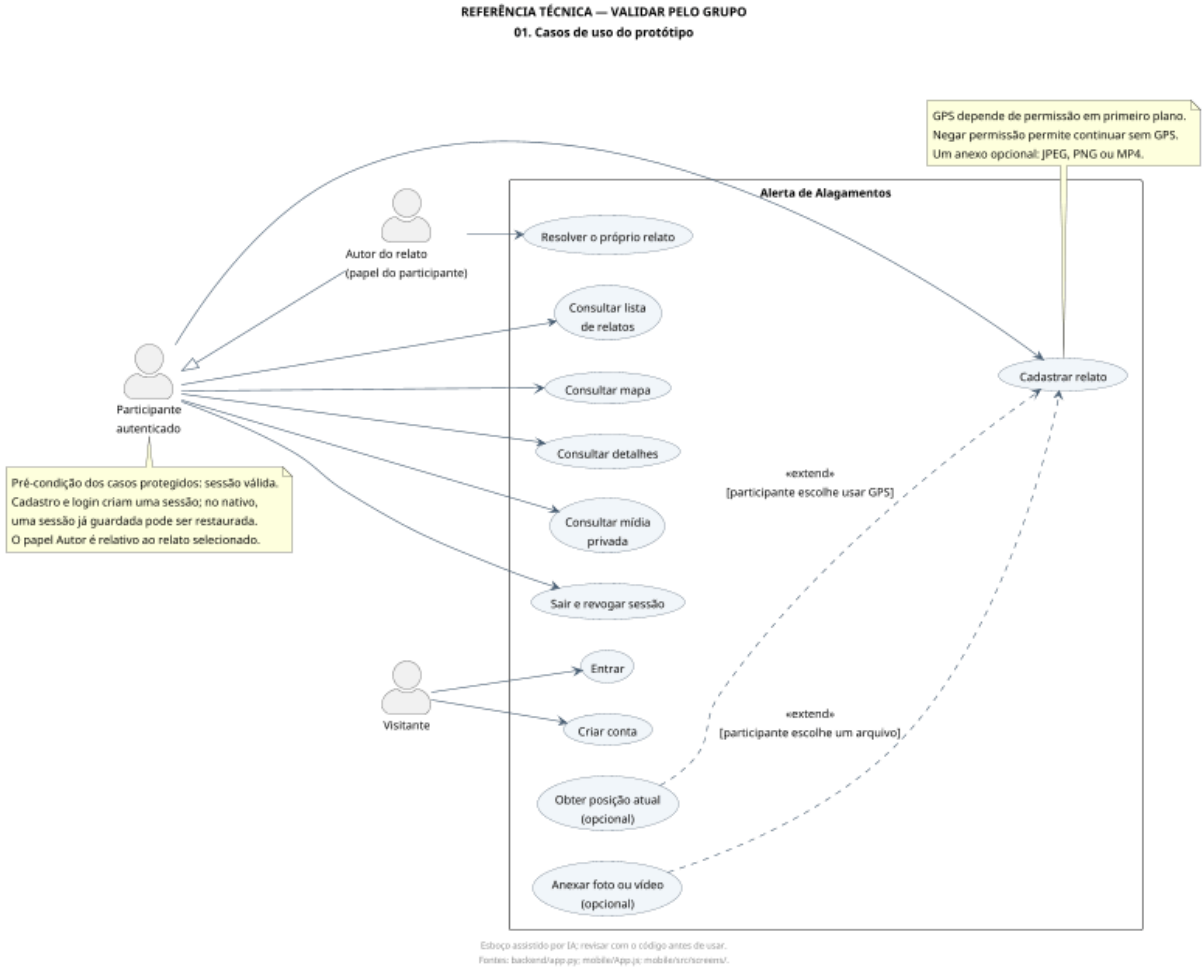
Requisito	**Módulo**	**Critério / evidência**
Conta e sessão	API e AuthScreen	Login válido; 401 sem sessão
Autoria	API e Detalhes	403 para outra conta
Relato	API e Cadastro	Campos válidos e persistência
GPS e mapa	LocalizacaoCampos e Mapas	Coordenadas válidas e fallback
Mídia privada	midia.py e AnexoViewer	Upload e acesso autenticado

Fonte: Modelo de planejamento, sem dados de integrantes.

5.2 Diagrama de casos de uso

A Figura 1 apresenta os atores e as funcionalidades disponibilizadas. O visitante cria conta ou entra; o participante autenticado consulta e cadastra. O papel de autor depende do relato selecionado e permite resolver o próprio registro. Localização e anexo ampliam o cadastro de forma opcional.

Figura 1 – Diagrama de casos de uso

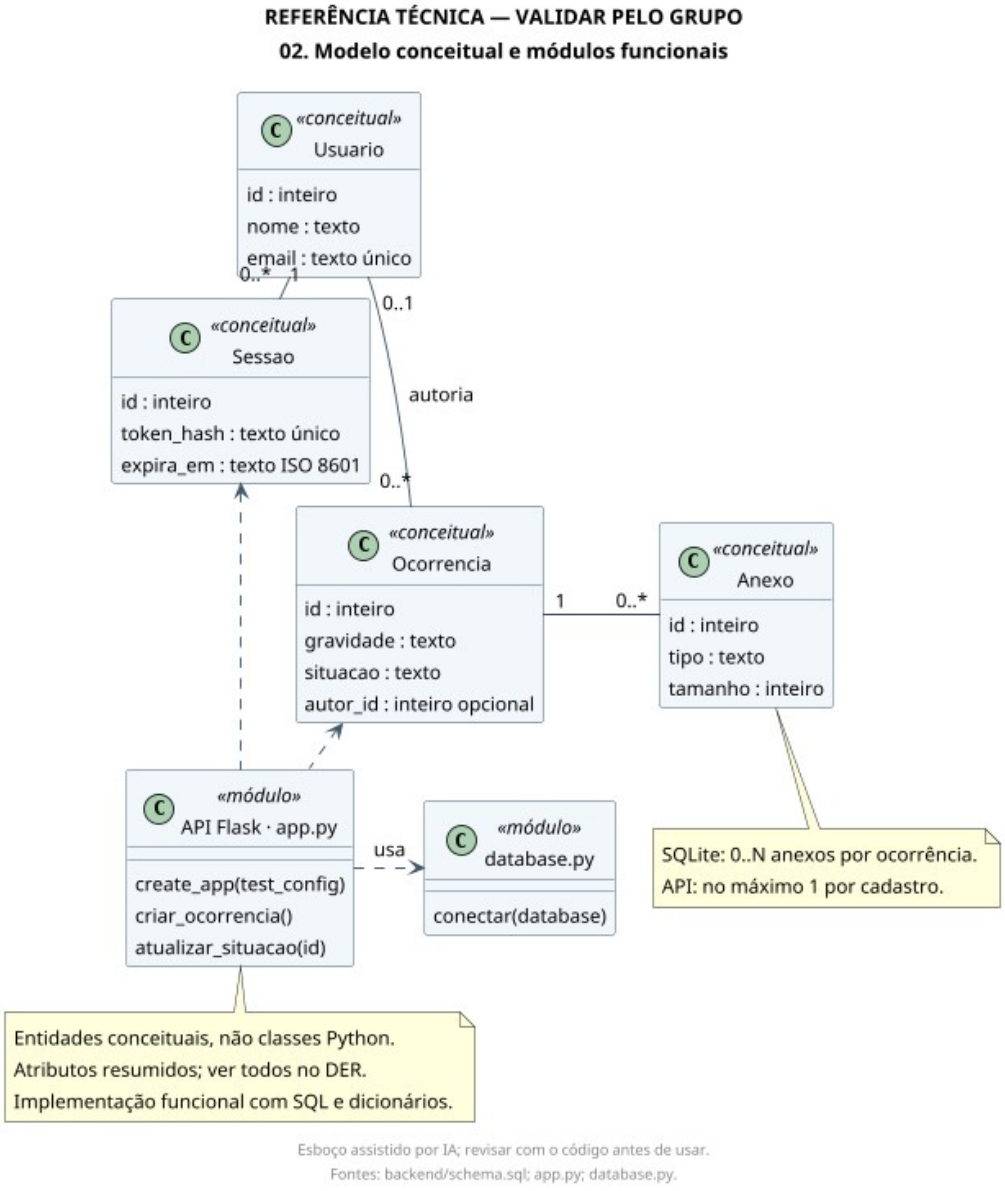


Fonte: elaboração assistida por IA, com base no código do projeto (028b704, 2026).
Nota: referência técnica assistida para validação do grupo; classes são conceituais, não classes Python. No DER, banco permite 0..N anexos e a API limita um por cadastro.

5.3 Diagrama de classes conceituais

A Figura 2 representa entidades conceituais e módulos funcionais. Usuário, sessão, ocorrência e anexo descrevem conceitos de dados. A implementação utiliza funções e SQL, sem classes Python correspondentes a cada entidade. As dependências indicam o acesso da API aos serviços de sessão e armazenamento.

Figura 2 – Diagrama de classes conceituais

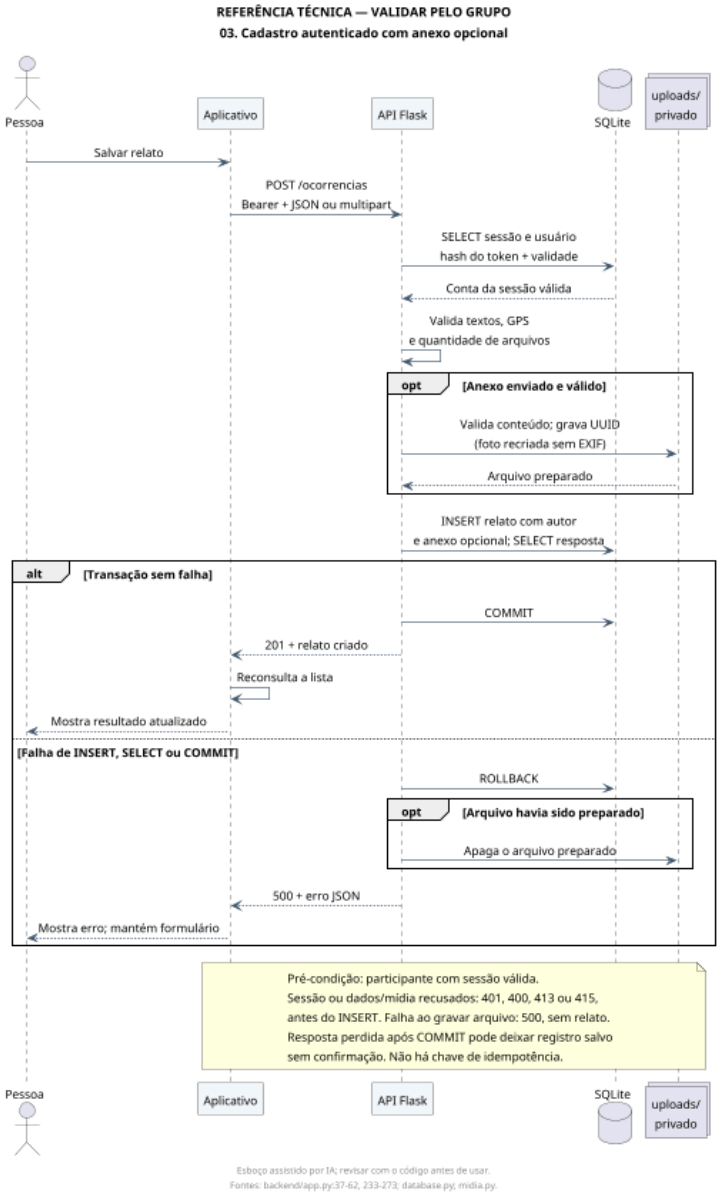


Fonte: elaboração assistida por IA, com base no código do projeto (028b704, 2026).
Nota: referência técnica assistida para validação do grupo; classes são conceituais, não classes Python. No DER, banco permite 0..N anexos e a API limita um por cadastro.

5.4 Diagrama de sequência

A Figura 3 descreve um cadastro autenticado. A API identifica a sessão, valida a entrada e prepara o arquivo opcional antes dos registros no banco. Após a confirmação, a resposta permite atualizar a lista. Uma falha de transação provoca rollback e remoção compensatória do arquivo preparado.

Figura 3 – Diagrama de sequência

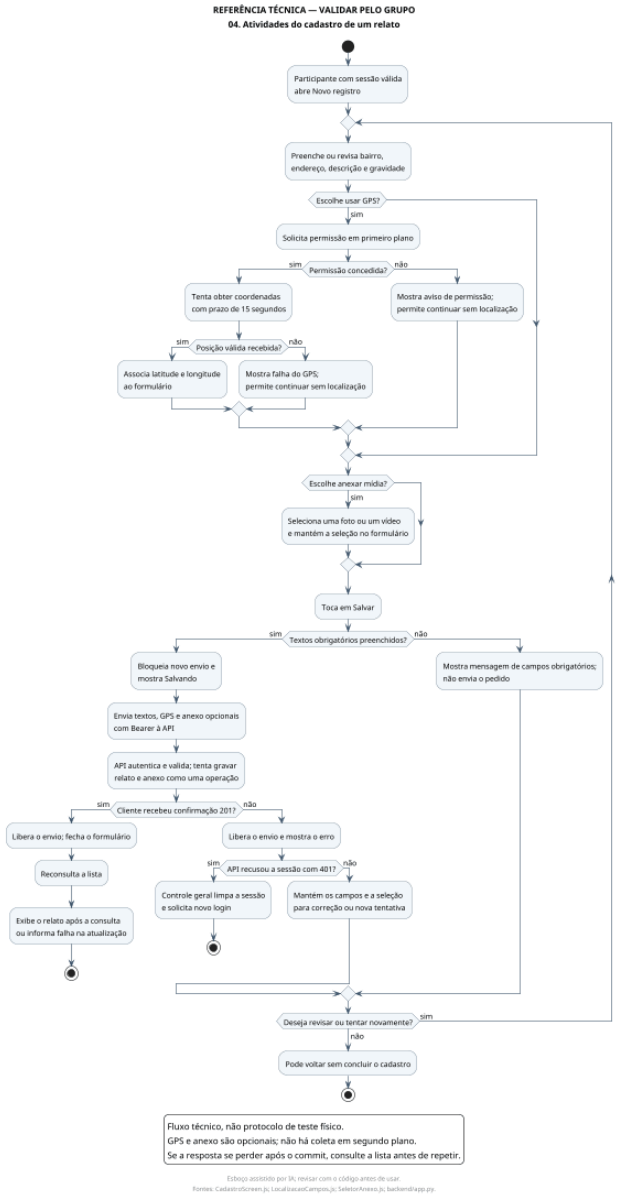


Fonte: elaboração assistida por IA, com base no código do projeto (028b704, 2026).
Nota: referência técnica assistida para validação do grupo; classes são conceituais, não classes Python. No DER, banco permite 0..N anexos e a API limita um por cadastro.

5.5 Diagrama de atividades

A Figura 4 apresenta as decisões do fluxo de cadastro. GPS e mídia são opcionais. Uma entrada inválida retorna ao formulário, enquanto uma sessão recusada exige novo acesso. Os estados de erro preservam os dados quando possível e o sucesso leva à consulta da lista atualizada.

Figura 4 – Diagrama de atividades

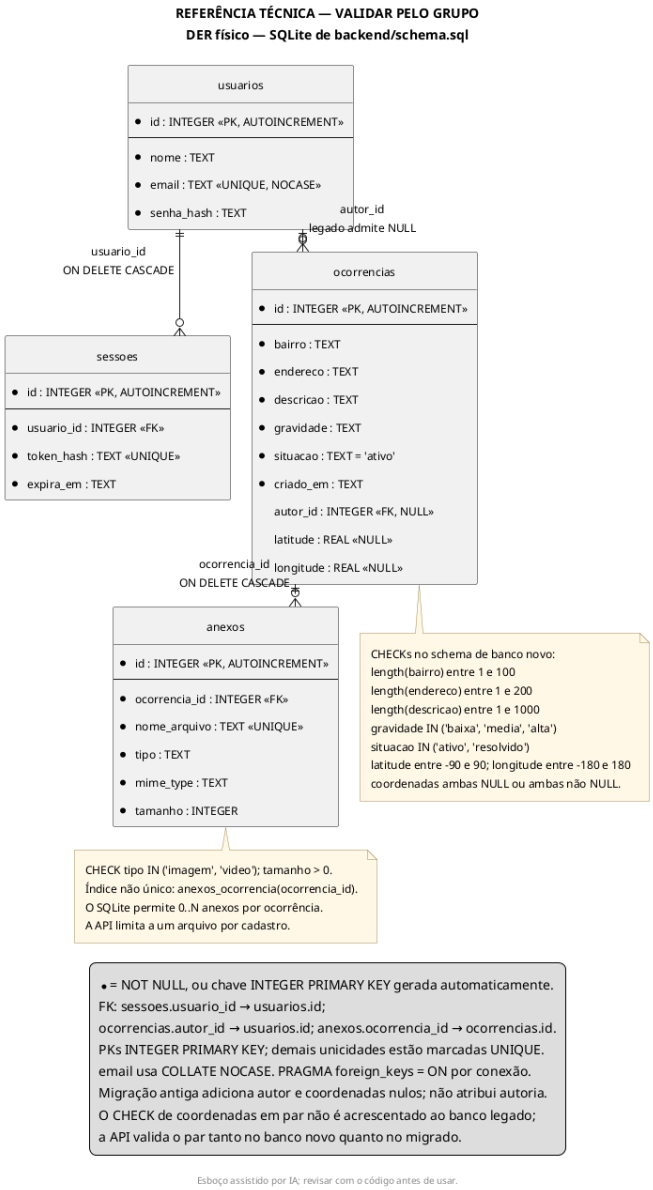


Fonte: elaboração assistida por IA, com base no código do projeto (028b704, 2026).
Nota: referência técnica assistida para validação do grupo; classes são conceituais, não classes Python. No DER, banco permite 0..N anexos e a API limita um por cadastro.

5.6 Diagrama entidade-relacionamento físico

A Figura 5 apresenta o modelo físico. As chaves estrangeiras relacionam sessões e relatos às contas e os metadados dos arquivos às ocorrências. O autor pode ser nulo nos dados legados. O banco permite vários anexos associados, enquanto a API restringe o cadastro atual a um arquivo.

Figura 5 – Diagrama entidade-relacionamento físico



Fonte: elaboração assistida por IA, com base no código do projeto (028b704, 2026).
Nota: referência técnica assistida para validação do grupo; classes são conceituais, não classes Python. No DER, banco permite 0..N anexos e a API limita um por cadastro.

6 RELATÓRIO DE CÓDIGO

Seleção técnica parcial, não é o fonte integral.

```
# Título: Alerta de Alagamentos
# Participantes: identificação omitida conforme solicitado.
# Referência: 028b704
```

Produção: 2.096 linhas físicas em 21 arquivos; inclui comentários e linhas vazias. As páginas seguintes selecionam 135 linhas existentes. Testes, configurações e arquivos gerados ficam separados.

Tabela 1 – Inventário de linhas físicas

Arquivo de produção	**Linhas**
backend/app.py	315
backend/autenticacao.py	23
backend/database.py	43
backend/init_db.py	17
backend/midia.py	169
backend/schema.sql	39
mobile/App.js	192
mobile/index.js	8
mobile/src/components/AnexoViewer.js	109
mobile/src/components/Interface.js	76
mobile/src/components/LocalizacaoCampos.js	77
mobile/src/components/MapaOcorrencias.native.js	68
mobile/src/components/MapaOcorrencias.web.js	107
mobile/src/components/SeletorAnexo.js	85
mobile/src/config.js	30
mobile/src/screens/AuthScreen.js	115
mobile/src/screens/CadastroScreen.js	123
mobile/src/screens/DetalhesScreen.js	134
mobile/src/screens/ListaScreen.js	155
mobile/src/services/api.js	162
mobile/src/theme.js	49

Fonte: contagem dos arquivos do commit 028b704 (2026).

Confirmar a aceitação de seleção legível + fonte completo separado. > indica quebra visual, sem alterar a linha original.

6.1 Trechos selecionados – parte 1

Foco a explicar pelo grupo: Conexão, chave estrangeira, commit, rollback e fechamento

Quadro 2 – Trecho de backend/database.py

```
 13 | def conectar(database):
 14 |     """Cada operação recebe sua conexão e confirma ou desfaz a
    > transação."""
 15 |     conexao = sqlite3.connect(str(database), timeout=10)
 16 |     conexao.row_factory = sqlite3.Row
 17 |     conexao.execute("PRAGMA foreign_keys = ON")
 18 |     try:
 19 |         yield conexao
 20 |         conexao.commit()
 21 |     except Exception:
 22 |         conexao.rollback()
 23 |         raise
 24 |     finally:
 25 |         conexao.close()
```

Fonte: código existente do projeto (028b704), backend/database.py, linhas 13–25.
A conexão é configurada com acesso por nome de coluna e chaves estrangeiras ativas. O bloco confirma a transação no sucesso, desfaz em exceção e fecha o recurso ao terminar.

Foco a explicar pelo grupo: SHA-256 do token e criação de sessão com validade

Quadro 3 – Trecho de backend/autenticacao.py

```
 12 | def hash_token(token):
 13 |     return hashlib.sha256(token.encode("utf-8")).hexdigest()
 14 |
 15 |
 16 | def criar_sessao(conexao, usuario_id):
 17 |     token = secrets.token_urlsafe(32)
 18 |     expira_em = (datetime.now(timezone.utc) +
    > timedelta(days=7)).isoformat(timespec="seconds")
 19 |     conexao.execute(
 20 |         "INSERT INTO sessoes(usuario_id, token_hash, expira_em)
    > VALUES (?, ?, ?)",
 21 |         (usuario_id, hash_token(token), expira_em),
 22 |     )
 23 |     return token, expira_em
```

Fonte: código existente do projeto (028b704), backend/autenticacao.py, linhas 12–23.
O token aleatório é entregue ao cliente, enquanto apenas seu hash é registrado. A validade de sete dias limita a sessão, e a consulta posterior usa o hash para identificar a conta.

6.2 Trechos selecionados – parte 2

Foco a explicar pelo grupo: Autenticação Bearer antes da operação

Quadro 4 – Trecho de backend/app.py

```
 49 |         partes = request.headers.get("Authorization", "").split()
 50 |         if len(partes) != 2 or partes[0].lower() != "bearer":
 51 |             return falha("Entre na sua conta para continuar.", 401)
 52 |         with conectar(app.config["DATABASE"]) as conexao:
 53 |             usuario = conexao.execute(
 54 |                 """SELECT u.id, u.nome, u.email FROM sessoes s
 55 |                    JOIN usuarios u ON u.id = s.usuario_id
 56 |                    WHERE s.token_hash = ? AND s.expira_em > ?""",
 57 |                 (hash_token(partes[1]), agora_utc()),
 58 |             ).fetchone()
 59 |         if usuario is None:
 60 |             return falha("Sessão inválida ou expirada. Entre
    > novamente.", 401)
 61 |         g.usuario = dict(usuario)
 62 |         g.token_hash = hash_token(partes[1])
```

Fonte: código existente do projeto (028b704), backend/app.py, linhas 49–62.

A requisição protegida precisa apresentar Bearer válido. A associação entre sessão e conta é consultada antes de disponibilizar a operação; uma sessão ausente ou vencida recebe 401.

Foco a explicar pelo grupo: Validação do par latitude e longitude

Quadro 5 – Trecho de backend/app.py

```
217 |         latitude, longitude = dados.get("latitude"),
    > dados.get("longitude")
218 |         if latitude is None and longitude is None:
219 |             valores.update(latitude=None, longitude=None)
220 |             return valores, None
221 |         for campo, valor, limite in (("latitude", latitude, 90),
    > ("longitude", longitude, 180)):
222 |             if isinstance(valor, bool) or (not multipart and not
    > isinstance(valor, (int, float))):
223 |                 return None, falha("Informe latitude e longitude
    > numéricas juntas.", 400)
224 |             try:
225 |                 numero = float(valor)
226 |             except (TypeError, ValueError, OverflowError):
227 |                 return None, falha("Informe latitude e longitude
    > numéricas juntas.", 400)
228 |             if not math.isfinite(numero) or not -limite <= numero <=
    >  limite:
229 |                 return None, falha(f"A {campo} deve estar entre
    > {-limite} e {limite}.", 400)
230 |             valores[campo] = numero
```

Fonte: código existente do projeto (028b704), backend/app.py, linhas 217–230.

Latitude e longitude são aceitas em conjunto. A validação rejeita tipos indevidos, valores não finitos e coordenadas fora dos intervalos, evitando posições inválidas no mapa.

6.3 Trechos selecionados – parte 3

Foco a explicar pelo grupo: INSERT parametrizado e remoção de arquivo se a transação falhar

Quadro 6 – Trecho de backend/app.py

```
250 |         try:
251 |             # Falha no INSERT, no anexo ou no commit desfaz o banco
    > e remove o arquivo.
252 |             with conectar(app.config["DATABASE"]) as conexao:
253 |                 cursor = conexao.execute(
254 |                     """INSERT INTO ocorrencias
255 |                        (bairro, endereco, descricao, gravidade,
    > situacao, criado_em, autor_id, latitude, longitude)
256 |                        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
257 |                     (valores["bairro"], valores["endereco"],
    > valores["descricao"], valores["gravidade"],
258 |                      "ativo", agora_utc(), g.usuario["id"],
    > valores["latitude"], valores["longitude"]),
259 |                 )
260 |                 identificador = cursor.lastrowid
261 |                 if anexo:
262 |                     conexao.execute(
263 |                         """INSERT INTO anexos(ocorrencia_id,
    > nome_arquivo, tipo, mime_type, tamanho)
264 |                            VALUES (?, ?, ?, ?, ?)""",
265 |                         (identificador, anexo["nome_arquivo"],
    > anexo["tipo"], anexo["mime_type"], anexo["tamanho"]),
266 |                     )
267 |                 registro = conexao.execute("SELECT * FROM
    > ocorrencias WHERE id = ?", (identificador,)).fetchone()
268 |                 resultado = serializar_ocorrencia(conexao, registro)
269 |         except Exception:
270 |             if anexo:
271 |                 apagar_arquivo(app.config["UPLOAD_FOLDER"],
    > anexo["nome_arquivo"])
272 |             raise
```

Fonte: código existente do projeto (028b704), backend/app.py, linhas 250–272.
Os valores são passados separadamente ao comando SQL. A ocorrência e seus metadados são confirmados na transação; a exceção também remove o arquivo preparado.

6.4 Trechos selecionados – parte 4

Foco a explicar pelo grupo: Autorização de alteração por autoria

Quadro 7 – Trecho de backend/app.py

```
292 |         with conectar(app.config["DATABASE"]) as conexao:
293 |             registro = conexao.execute("SELECT * FROM ocorrencias
    > WHERE id = ?", (id,)).fetchone()
294 |             if registro is None:
295 |                 return falha("Ocorrência não encontrada.", 404)
296 |             if not pode_alterar(registro):
297 |                 return falha("Somente quem cadastrou pode alterar
    > esta ocorrência.", 403)
298 |             conexao.execute("UPDATE ocorrencias SET situacao = ?
    > WHERE id = ?", (situacao.strip(), id))
299 |             registro = conexao.execute("SELECT * FROM ocorrencias
    > WHERE id = ?", (id,)).fetchone()
300 |             resultado = serializar_ocorrencia(conexao, registro)
301 |         return jsonify(resultado), 200
```

Fonte: código existente do projeto (028b704), backend/app.py, linhas 292–301.

A autorização compara a conta da sessão com o autor antes do UPDATE. Outra conta pode consultar o registro compartilhado, mas recebe 403 ao tentar resolver a ocorrência.

Foco a explicar pelo grupo: Orientação e recriação de imagem sem EXIF

Quadro 8 – Trecho de backend/midia.py

```
 38 |             with Image.open(BytesIO(conteudo)) as foto:
 39 |                 foto.load()
 40 |                 with ImageOps.exif_transpose(foto) as orientada:
 41 |                     modo = "RGB" if formato == "JPEG" else "RGBA"
 42 |                     with orientada.convert(modo) as convertida:
 43 |                         # Uma imagem nova elimina EXIF, GPS e os
    > demais metadados.
 44 |                         with Image.new(modo, convertida.size) as
    > limpa:
 45 |                             limpa.paste(convertida)
 46 |                             saida = BytesIO()
 47 |                             limpa.save(saida, format=formato,
    > **({"quality": 85} if formato == "JPEG" else {}))
 48 |                             resultado = saida.getvalue()
```

Fonte: código existente do projeto (028b704), backend/midia.py, linhas 38–48.

A imagem é orientada e recriada a partir dos pixels. O arquivo resultante não conserva os metadados EXIF da imagem de entrada, incluindo sua possível localização.

6.5 Trechos selecionados – parte 5

Foco a explicar pelo grupo: Fetch com Bearer, AbortController e multipart

Quadro 9 – Trecho de mobile/src/services/api.js

```
 49 |     resposta = await fetch(`${API_URL}${caminho}`, {
 50 |       ...opcoes,
 51 |       headers: {
 52 |         Accept: arquivo ? '*/*' : 'application/json',
 53 |         ...(!publica && token ? { Authorization: `Bearer ${token}` }
    >  : {}),
 54 |         ...(opcoes.body && !formulario ? { 'Content-Type':
    > 'application/json' } : {}),
 55 |       },
 56 |       signal: controlador.signal,
 57 |     });
 58 |     // No multipart, o próprio cliente define Content-Type com seu
    > boundary.
 59 |     if (arquivo && resposta.ok) return await resposta.blob();
```

Fonte: código existente do projeto (028b704), mobile/src/services/api.js, linhas 49–59.
A função centraliza o envio da sessão e o cancelamento por prazo. No multipart, o cliente define o boundary; a resposta precisa ser verificada antes de indicar sucesso.

Foco a explicar pelo grupo: Permissão em primeiro plano e GPS com prazo

Quadro 10 – Trecho de mobile/src/components/LocalizacaoCampos.js

```
 27 |       // A permissão só é solicitada após o toque. Não há
    > rastreamento em segundo plano.
 28 |       const permissao = await
    > Location.requestForegroundPermissionsAsync();
 29 |       if (!permissao.granted) {
 30 |         throw new Error('Localização não autorizada. Você pode
    > continuar sem GPS e informar o endereço.');
 31 |       }
 32 |       if (!montada.current || bloqueado.current) return;
 33 |       /** @type {Promise<never>} */
 34 |       const prazo = new Promise((_, rejeitar) => {
 35 |         temporizador = setTimeout(() => rejeitar(new Error('O GPS
    > demorou para responder. Tente novamente ou continue sem
    > localização.')), 15000);
 36 |       });
 37 |       const local = await Promise.race([
 38 |         Location.getCurrentPositionAsync({ accuracy:
    > Location.Accuracy.Balanced }), prazo,
 39 |       ]);
 40 |       const { latitude, longitude } = local.coords;
 41 |       if (!Number.isFinite(latitude) || !Number.isFinite(longitude))
    >  {
 42 |         throw new Error('O dispositivo não informou uma localização
    > válida.');
 43 |       }
 44 |       // Uma resposta atrasada não pode alterar um formulário que já
    >  foi enviado.
 45 |       if (montada.current && !bloqueado.current) onChange({
    > latitude, longitude });
```

Fonte: código existente do projeto (028b704), mobile/src/components/LocalizacaoCampos.js, linhas 27–45.
A permissão é solicitada por ação do participante e a espera tem prazo. O resultado não deve alterar uma tela desmontada ou um formulário que já esteja bloqueado para envio.

6.6 Trechos selecionados – parte 6

Foco a explicar pelo grupo: Marcadores nativos com coordenadas dos relatos

Quadro 11 – Trecho de mobile/src/components/MapaOcorrencias.native.js

```
 42 |           {pontos.map((item) => (
 43 |             <Marker
 44 |               key={item.id} coordinate={{ latitude: item.latitude,
    > longitude: item.longitude }}
 45 |               title={item.bairro} description={`Gravidade relatada:
    > ${rotuloGravidade(item.gravidade)}`}
 46 |               pinColor={item.gravidade === 'alta' ? '#A73535' :
    > item.gravidade === 'media' ? '#B2750B' : cores.verde}
 47 |               onPress={() => onAbrirDetalhes?.(item.id)}
 48 |             />
 49 |           ))}
```

Fonte: código existente do projeto (028b704), mobile/src/components/MapaOcorrencias.native.js, linhas 42–49.

O mapa apresenta somente as coordenadas existentes nos relatos. O marcador usa os dados retornados pela API e oferece acesso ao registro correspondente.

7 CONSIDERAÇÕES FINAIS

O protótipo implementa o fluxo de relatos com autenticação, autoria, localização e mídia opcionais. Os testes documentados demonstram as operações locais exercitadas, a interação entre duas contas web e a persistência de dados e arquivos após reiniciar o serviço. A regra de alteração exclusiva pelo autor é aplicada no servidor e refletida pela interface.

A avaliação deve considerar as limitações observadas: armazenamento e API centralizados, atualização manual, dependência de rede e de provedores de mapas, ausência de moderação e de operação offline. Os testes web utilizaram coordenadas simuladas e não substituem os recursos de celulares físicos. O critério móvel final depende do ensaio em dois aparelhos com Expo Go e do registro de seus resultados.

A organização em cliente, API e armazenamento permite relacionar a implementação aos conceitos de comunicação entre processos, representação de dados, autenticação, autorização e transações. A continuidade do trabalho deve priorizar a validação física e a preparação da demonstração, conservando a rastreabilidade entre requisito, código e evidência. Informações acadêmicas individuais permanecem sujeitas à confirmação de cada participante.

7.1 Apresentação do programa

Todos os integrantes devem participar. A rubrica informa perda de 50% para quem não participar. Registrar apenas participação real.

Quadro 12 – Divisão do ensaio a preencher

Etapa a ensaiar	**Participante / tempo real**
Objetivo e arquitetura	
Demonstração no computador e Expo Go	
Duas contas / dispositivos	
GPS, mapa, imagem e vídeo	
Autoria e persistência	

Fonte: Modelo de planejamento, sem dados de integrantes.

O teste físico em dois celulares permanece pendente. Ver APRESENTACAO.md e TESTES.md.

REFERÊNCIAS

As referências técnicas abaixo identificam documentos e o código que fundamentam a descrição. As referências externas devem ser conferidas pelo grupo; não foi atribuída uma data de consulta aos integrantes.

FIELDING, Roy T.; NOTTINGHAM, Mark; RESCHKE, Julian. HTTP Semantics. IETF, 2022. RFC 9110. DOI: 10.17487/RFC9110.

BRAY, Tim. The JavaScript Object Notation (JSON) Data Interchange Format. IETF, 2017. RFC 8259. DOI: 10.17487/RFC8259.

MASINTER, Larry. Returning Values from Forms: multipart/form-data. IETF, 2015. RFC 7578. DOI: 10.17487/RFC7578.

ALERTA DE ALAGAMENTOS. Código-fonte do projeto. GitHub, 2026. Versão 028b7041702f68e1a473d57acbe95037a8341a79. Disponível em: https://github.com/matheuslagares/APS_8. Base técnica inspecionada e identificada nos anexos.

Não declarar leitura de fontes não consultadas. Ver NORMAS_ABNT_ADOTADAS.md para as edições e escolhas de formatação utilizadas no modelo.

APÊNDICE A – ORIENTAÇÕES PARA FICHAS DE APS

Esta orientação não substitui a ficha oficial. Anexar uma ficha por aluno, com atividades cronológicas, horas reais, total e assinatura manuscrita. Não foram atribuídas horas nem produzidas assinaturas.

Quadro 13 – Conferência auxiliar das atividades

Data	Atividade real	Horas	Evidência

Fonte: Modelo de planejamento, sem dados de integrantes.

As fichas oficiais externas podem integrar um anexo, conforme a orientação institucional. Não apresentar este quadro auxiliar como formulário oficial.